

华东师范大学

EAST CHINA NORMAL UNIVERSITY

基于uCore OS的 多级反馈队列调度器与智能行为预测实现

华东师范大学 | T202510269998251 | 3FW | 操作系统原理赛道（模块实验设计）

队员：吴昱阳、王业涵、符浩峰
指导教师：石亮

2026/01/17

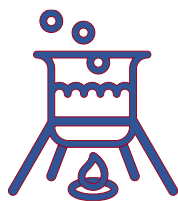
CONTENTS

目录



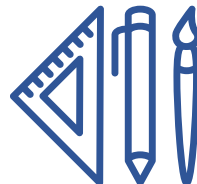
项目背景与意义

Research background



运行结果分析

Result analysis



项目思路与实现

Project ideas and methods



项目总结

Conclusion and prospect



项目背景

传统算法痛点

- RR 轮转调度：响应慢，无法区分进程类型，I/O密集型进程体验差。
- 传统MLFQ：存在“欺骗调度器”漏洞——恶意进程通过在时间片结束前主动 yield 来长期霸占高优先级队列。

基准代码

清华大学uCore OS lab6_result:

- 算法原型：Stride Scheduling (步进调度算法)。
- 核心实现：使用Skew Heap (斜堆) 维护运行队列，实现高效调度。
- 调度特性：基于优先级的比例份额调度，相比简单轮转 (RR) 更高效。

项目背景与意义

项目思路与实现

运行结果分析

项目总结



项目意义

创新目标

设计一个既能保证交互响应，又能抵抗恶意欺骗的内核级调度器。

项目背景与意义

项目价值

抗恶意 (Anti-Gaming): 识别“假装I/O”的CPU密集型进程。

自适应 (Adaptive): 动态调整优先级，平衡吞吐量与响应时间。

教育性: 将机器学习思想 (EWMA) 引入OS内核设计。

项目思路与实现

运行结果分析

项目总结



MLFQ (Multi-Level Feedback Queue)

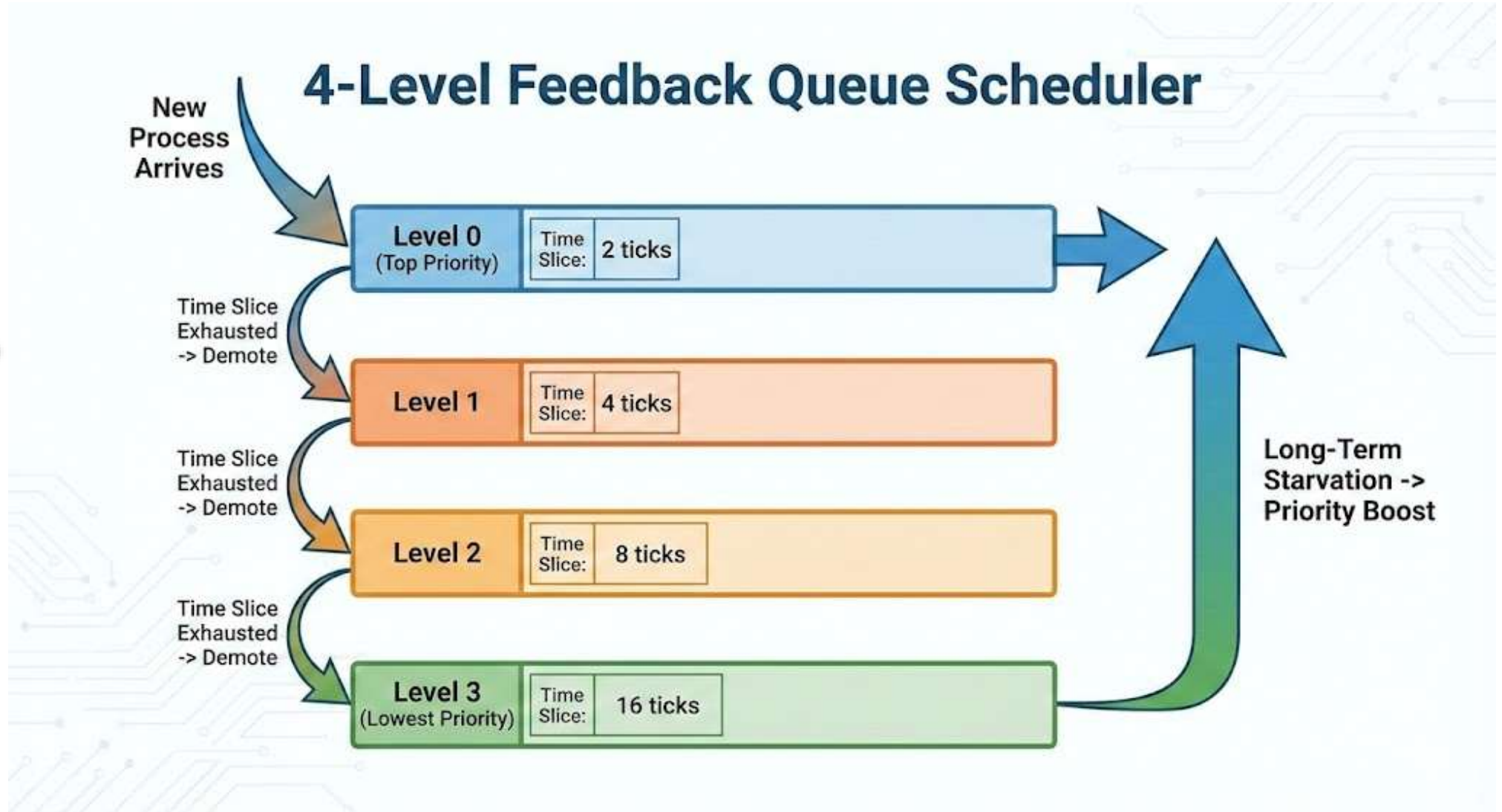
05

项目背景与意义

项目思路与方法

运行结果分析

项目总结





内核数据结构扩展

06

● kern/process/proc.h

```
67      /* ===== MLFQ & 智能预测扩展字段 ===== */
68
69      /* 1. MLFQ基础状态 */
70      int mlfq_level;                /* 当前所在的队列层级 */
71      int current_slice_ticks;       /* 当前时间片剩余ticks数 */
72
73      /* 2. 行为特征统计（用于智能预测） */
74      int sched_count;               /* 被调度次数 */
75      unsigned int cpu_time_total;   /* 总CPU运行时间(ticks) */
76      unsigned int last_burst;       /* 上一次CPU占用时间(ticks) */
77
78      /* 3. 预测评分（定点数：0-1000对应0.0-1.0） */
79      int cpu_score;                 /* CPU密集程度评分（越高表示越CPU密集型） */
80
81      /* ===== MLFQ扩展结束 ===== */
```

项目背景与意义

项目思路与方法

运行结果分析

项目总结



智能预测算法 (EWMA)



kern/schedule/mlfq_sched.c

项目背景与意义

项目思路与方法

运行结果分析

项目总结

$\text{NewScore} = 0.6 * \text{OldScore} + 0.4 * \text{CurrentRatio}$, 其中

OldScore 为更新前的 `proc->cpu_score`;

NewScore 为更新后的 `proc->cpu_score`;

CurrentRatio 为本次 CPU 使用率。

```
int ratio = (used * SCORE_SCALE) / max_slice; // 计算时间片使用比率 (0-1000)
```

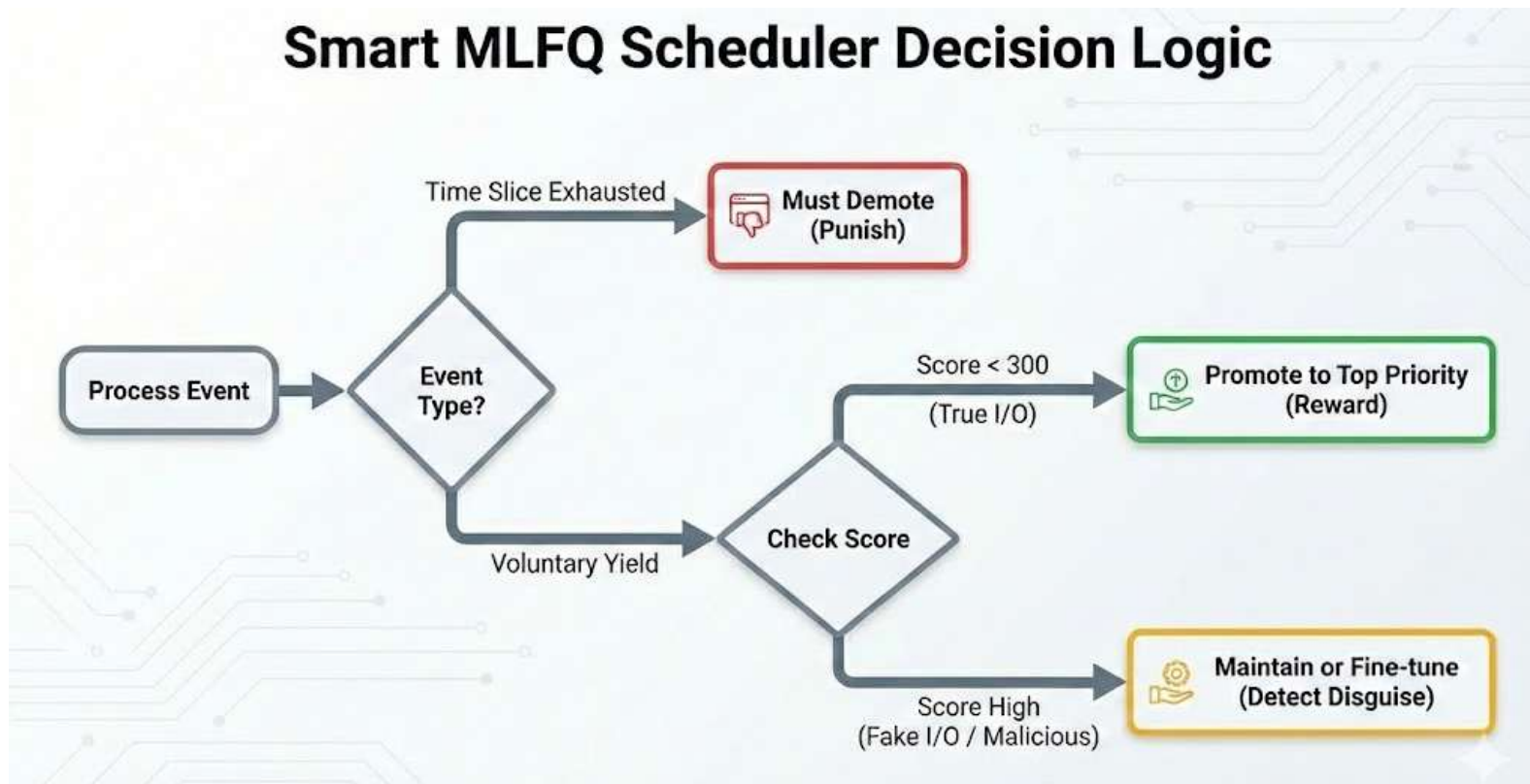
```
if (proc->cpu_score == 0)    proc->cpu_score = ratio; // 第一次直接赋值为ratio
```

```
else proc->cpu_score = (6 * proc->cpu_score + 4 * ratio) / 10; // 计算cpu_score
```



智能调度决策逻辑

Smart MLFQ Scheduler Decision Logic



防饥饿机制 (Priority Boost)

- 机制：全局计时器 `global_timer` 达到 `BOOST_TICKS` (1000) 时。
- 动作：Reset all to Level 0。

项目背景与意义

项目思路与方法

运行结果分析

项目总结



RR调度器分析

· 现象：I/O密集型进程（PID 4, 7）的周转时间高达 310ms。与CPU密集型进程（PID 3, 5）的 293ms 几乎没有区别。

· 结论：IO密集型进程体验差。

```
[RESULT] PID: 5 | Type: CPU-Heavy | Resp: 5 ms | Turnaround: 293 ms
```

```
[Kernel Stat] PID 5 Exit Info:
```

```
- Context Switches: 29  
- CPU Occupied Time: 140 ticks  
(PID 2) CPU Phase 70/80
```

```
[RESULT] PID: 3 | Type: CPU-Heavy | Resp: 0 ms | Turnaround: 296 ms
```

```
[Kernel Stat] PID 3 Exit Info:
```

```
- Context Switches: 30  
- CPU Occupied Time: 148 ticks  
(PID 6) Mix Phase 40/80  
(PID 4) I/O Yield 40/80  
(PID 7) I/O Yield 40/80  
(PID 6) Mix Phase 60/80  
(PID 4) I/O Yield 60/80  
(PID 7) I/O Yield 60/80  
(PID 6) Mix Phase 70/80
```

```
[RESULT] PID: 6 | Type: Mixed-Work | Resp: 10 ms | Turnaround: 310 ms
```

```
[Kernel Stat] PID 6 Exit Info:
```

```
- Context Switches: 69  
- CPU Occupied Time: 22 ticks  
(PID 4) I/O Yield 70/80
```

```
[RESULT] PID: 4 | Type: IO-Heavy | Resp: 5 ms | Turnaround: 310 ms
```

```
[Kernel Stat] PID 4 Exit Info:
```

```
- Context Switches: 81  
- CPU Occupied Time: 1 ticks  
(PID 7) I/O Yield 79/80
```

```
[RESULT] PID: 7 | Type: IO-Heavy | Resp: 12 ms | Turnaround: 311 ms
```



Basic MLFQ分析

- 现象：I/O 进程的周转时间从 310ms 降至 12ms；调度逻辑的单一（只要 Yield 就回 Level 0）。

隐患：机制过于简单粗暴，混合型进程 PID 6 只要一 Yield 就立即回到 Top。

```
[Basic MLFQ] PID: 4 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 7 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 4 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 7 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 4 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 7 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 4 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 7 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 4 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 7 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 4 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 7 | Level: 0 (Logic: Timeout->Down, Yield->Top)
(PID 4) I/O Yield 79/80
```

```
[RESULT] PID: 4 | Type: IO-Heavy | Resp: 2 ms | Turnaround: 12 ms
```

(PID 6) Mix Phase 40/80

```
[Basic MLFQ] PID: 6 | Level: 1 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 1 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
[Basic MLFQ] PID: 6 | Level: 0 (Logic: Timeout->Down, Yield->Top)
```



Smart MLFQ分析

- 性能表现：I/O 高效响应的同时，对混合型任务的处理也更有效率。

- 智能预测机制（以PID 6为例）：

阶段1：高分降级。进程刚开始计算，Score 高达1000，被识别为 CPU 密集，降级到 Level 1；

阶段2：动态评分下降。进程开始进行 I/O 操作，Score 逐渐下降；

阶段3：动态调整优先级。即使当前 Score 很低，调度器持续监测。当 Score 回升时，调度器会试探性地降级，体现了算法的自适应性。

```
[MLFQ PRED] PID: 4 | Used: 0/2 | Score: 5 | Level: 0 -> 0  
[MLFQ PRED] PID: 7 | Used: 0/2 | Score: 7 | Level: 0 -> 0  
(PID 4) I/O Yield 79/80
```

```
[RESULT] PID: 4 | Type: IO-Heavy | Resp: 2 ms | Turnaround: 13 ms
```

```
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 17 | Level: 0 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 10 | Level: 0 -> 0  
(PID 6) Mix Phase 79/80
```

```
[RESULT] PID: 6 | Type: Mixed-Work | Resp: 4 ms | Turnaround: 47 ms
```

```
(PID 6) [Mixed-Work] Started. Resp: 4 ms  
(PID 6) Mix Phase 0/80
```

```
[MLFQ PRED] PID: 6 | Used: 2/2 | Score: 1000 | Level: 0 -> 1
```

```
[MLFQ PRED] PID: 6 | Used: 1/4 | Score: 700 | Level: 1 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 420 | Level: 0 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 252 | Level: 0 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 151 | Level: 0 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 90 | Level: 0 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 54 | Level: 0 -> 0  
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 32 | Level: 0 -> 0
```

```
[MLFQ PRED] PID: 6 | Used: 0/2 | Score: 19 | Level: 0 -> 0
```

```
[MLFQ PRED] PID: 6 | Used: 2/2 | Score: 411 | Level: 0 -> 1
```

项目背景与意义

项目思路与方法

运行结果分析

项目总结



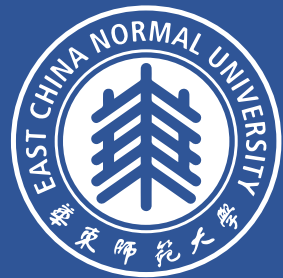
项目总结

- 算法设计：验证了跨领域思想（机器学习 + 操作系统）的可行性；即使是简单的EWMA算法也能在系统底层中发挥重要作用。
- 工程运用：体会到了内核调试的难度 (Kernel Panic)，锻炼了系统编程能力。
- 未来展望：可以引入动态调整机制，使平滑系数 α 自适应调优；可以从更多方面考虑，综合改进MLFQ调度性能。

团队分工

- 吴昱阳：Smart MLFQ、EWMA等核心算法实现。
- 王业涵：三种算法测试对比、答辩PPT制作。
- 符浩峰：实验指导书等相关说明文档撰写。





華東師範大學
EAST CHINA NORMAL UNIVERSITY

Q & A

华东师范大学 | T202510269998251 | 3FW | 操作系统原理赛道（模块实验设计）

组员：吴昱阳、王业涵、符浩峰

指导教师：石亮